

REAL ESTATE PRICE PREDICTION



Project ID: UE-CS-13

Project Advisor: Dr. Muhammad Asif

Submitted By

Zohaib Akhtar

BSF2101437

University of Education

REAL ESTATE PRICE PREDICTION
“A Predictive Model and Web Application for Accurate
Housing Valuation”

BS in Computer Science-2025

A project submitted in partial fulfillment of the requirements
for the award of the degree of
BS in Computer Science

ATTOCK CAMPUS
UNIVERSITY OF EDUCATION
LAHORE

“I hereby declare that I have read this project documentation and in my opinion this project is sufficient in terms of scope and quality for the award of the degree of BS in Computer Science.”

Project Primary Supervisor
Dr. Muhammad Asif
Assistant Professor
(Computer Science)
University of Education Lahore,
Attock Campus

Project Examiner
Muhammad Umar Farooq
Lecturer
University of Education Lahore,
DG Khan Campus

DECLARATION

I declare that this project title entitled “**REAL ESTATE PRICE PREDICTION**” is the result of my own research and development except as cited in the references. This project has not been accepted for any degree and is not concurrently submitted in candidate for any other degree. At any time if my statement is found to be incorrect even afterwards of BS in Computer Science, the university has the right to withdraw my BS in Computer Science degree.

Signature: _____

Signature: _____

Name: _____

Name: _____

Date: _____

Date: _____

Signature: _____

Signature: _____

Name: _____

Name: _____

Date: _____

Date: _____

PLAGIARISM UNDERTAKEN

I solemnly declare that project work presented in this documentation entitles “**REAL ESTATE PRICE PREDICTION**” is solely my work with no significant contribution from any other person. Small contribution/help wherever taken has been acknowledged and that complete project has been written by me.

I understand that zero tolerance policy of the HEC and University of Education, Lahore towards plagiarism. Therefore, we as an author of the above titled project declare that no portion of my project documentation and any material used as reference is properly referred/ cited.

I undertake that if I am found guilty of any formal plagiarism in the above titled project even after award of BS degree, the University reserve the rights to withdraw/revoke my BS degree and that HEC and the University has the right to publish my name on the HEC/University Website on which names of students are placed who submitted plagiarized projects.

Signature: _____

Name: _____

Name: _____

Date: _____

Date: _____

Signature: _____

Signature: _____

Name: _____

Name: _____

Date: _____

Date: _____

Signature: _____

CERTIFICATE OF APPROVAL

This is to certify that the project work presented in this documentation entitled, “**REAL ESTATE PRICE PREDICTION**”, was conducted by “**Zohaib Akhtar**”, under the supervision of “**Dr. Muhammad Asif**”. No part of this project has been submitted anywhere else for any degree. This project is submitted to the “**Attock Campus, University of Education**” is partial fulfillment of the requirements of the degree of BS in Computer Science.

Signature: _____

ZOHAIB AKHTAR

Name: _____

May 27, 2025

Date: _____

Project Primary Supervisor
Dr. Muhammad Asif
Assistant Professor
(Computer Science)
University of Education Lahore,
Attock Campus

Project Examiner
Muhammad Umar Farooq
Lecturer
University of Education Lahore,
DG Khan Campus

**OFFICE OF CONTROLLER OF EXAMINATION
NOTIFICATION**

No: _____

Date: _____

It is notified for the nomination of all the concerned that Mr./Ms. (Zohaib Akhtar) , BS student of Department of Information Sciences of University of Education, Lahore Attock Campus has completed all the requirements for the award of BS Degree in the discipline of Computer Science as per detail given hereunder:

BS in Computer Science		Cumulative Result			
		Credit Hours: _____			Cumulative Grade Point Average (CGPA)
Registration No	Complete Name	Course work	Project	Total	

Project Title: _____

Name of Supervisor: _____

**Signed by
Inter Controller of Examination**

ACKNOWLEDGEMENT

We truly acknowledge the cooperation and help make by Name of **Dr. Muhammad Asif (Assistant Professor, Supervisor) of University of Education Lahore, Attock Campus**. He has been a constant source of guidance throughout the course of this project. We would also like to thank **Mrs. Sidra Ijaz** from **Lecturer, University of Education Lahore, Attock Campus** for his help and guidance throughout this project. We are also thankful to our friends and families whose silent support led us to complete our project.

1- Dr. Muhammad Asif

2- Mrs. Sidra Ijaz

Date:

Table of Contents

CHAPTER 1	1
GATHERING & ANALYZING INFORMATION	1
1.1 Introduction	2
1.2 Problem Statement	3
1.3 Goals & Objectives	3
1.4 Research Questions and Their Answers	3
1.5 Related Work	4
1.6 Methodology	6
1.6.1 Available Methodologies.....	6
1.6.2 Chosen Methodology	7
1.6.3 Reasons for Chosen Methodology	8
1.7 Definitions, Acronyms, and Abbreviations.....	9
CHAPTER NO. 2	11
Software Requirement Specification(SRS).....	11
2.1 Stakeholders Characteristics	12
2.1.1 Primary Stakeholders.....	12
2.1.2 Secondary Stakeholders	12
2.2 Domain Requirements	13
2.3 Functional Requirements	13
2.4 Non-Functional Requirements.....	14
Analysis [Use Case Description and Use Case Model].....	16
3.1 Use Case: Load Raw Data.....	17
3.2 Use Case: Apply Preprocessing Pipeline.....	18
3.3 Use Case: Train House Price Prediction Model.....	20
3.4 Use Case: Save Trained Model	21
3.5 Use Case: Load Trained Model	22
3.6 Use Case: Input Property Details	23
3.7 Use Case: Generate House Price Prediction.....	24
3.8 Use Case: Display Predicted House Price	26
3.9 Use Case Model.....	28
CHAPTER 4.....	29
Design[with Description of each diagram].....	29
4.1 Architecture Diagram.....	30

4.3 Data Flow Diagram (Level 0 and 1)	32
4.3.1 Level 0 Data Flow Diagram.....	32
4.3.2 Level 1 Data Flow Diagram:	33
4.4 Class Diagram	34
4.5 Sequence Diagram (for Use Case: Predict House Price)	35
CHAPTER 5.....	37
Graphical User Interfaces	37
5.1 Mockups of working software application.....	38
5.2 User Interaction Workflow	40
CHAPTER 6	42
Testing.....	42
6.1 Introduction.....	43
6.2 Test Scenario	43
6.2.1 Test Case : Predict House Price with Valid Inputs.....	43
6.2.2 Test Case : Predict House Price with Edge Case Ocean Proximity ('ISLAND')	44
6.3 Test Plan	46
6.4 Definition of Test Cases	47
6.5 Test Cases Specifications.....	47
6.5.1 Black Box Test Cases and their Results	47
6.5.2 White Box Test Cases and their Results	49
CHAPTER 7	52
Conclusion and Future work	52
7.1 Conclusion	53
7.2 Future work.....	53
References.....	54

Abstract

A model built for predicting house prices based on the California housing data. The aim is to pre-process some complex data founding from the real estate market with different features types, missing data, and skewed distribution in order to train a robust predict model and integrate it into an end-to-end application. The crux is that it's a great resource for guessing property values, based on attributes & location. The methodology used to perform this analysis was pedagogical and guided through initial EDA to understand the structure of the dataset, to assess (e.g., identify 207 entries) for missing values (of 'total_bedrooms'), and further verify basic statistics. We then conducted more advanced EDA using extreme value analysis, such as feature distribution analysis (heavy tailed and multi-modal distribution analysis on housing prices), and correlation analysis (to see which features were positively/negatively associated with the dependent variable). Median_income was strongly positively correlated (~ 0.688) with median_house_value, while latitude and an engineered bedrooms_ratio had modest to weak negative correlations.

The data was split into training and test set (80-20 split) with stratified sampling on ocean_proximity to ensure geographical spread. Also, missing data was imputation. Numerical attributes were scaled and the categorical feature 'ocean_proximity' was one hot encoded. Many features were added by feature engineering (computing the ratio of the numbers of rooms, the number of bedrooms, etc) log-transforming continuous features (such as population density and household income) and working with geographical data by: k-means clustering the latitude and longitude to create geographic similarity features, creating a custom similarity feature (housing age). Special transformer classes were also developed to make the pipeline more flexible. The model selection considered several regression processes. A RandomForestRegressor was finally selected following the evaluation of feature importance and cross-validation. The Model is trained with pre-processed training data with certain set of hyper parameters then save to deploy. The experimental results demonstrate the effectiveness of the proposed pipeline and model. It performs well on test set with R^2 of 0.8198 and Root Mean Squared Error (RMSE) of about \$47,537. This means, the Model do a good job in accounting for 82% of the variance of house prices. The fine-tuning, particularly with respect to the engineered features and stratified sampling, was crucial in that regard.

In summary, this task achieved an end-to-end data preprocessing and machine learning pipeline for California house pricing prediction. The path from exploratory analysis to sophisticated feature engineering and model building produced a model with strong performance. The results of this study are very practical with potential for deployment in household value estimation. Feature importance and correlation analysis should be used to inform real estate decisions. So in future, each of us could use the same preprocess pipeline to deal with the similar dataset. What's more, the implementation of the model in a simple web application, as we can see, shows a realistic way to be applied in the real word. However, the project is not without its problems. Treatment of outliers and heavy-tailed distributions is delicate, because aggressive capping or transforming will result in loss of information. Engineered features such as geographical clusters or age similarity can be effective but are parameter dependent and need fine tuning. For some features there was certain multicollinearity observed that may affect model interpretation. Although it works to illustrate deployment, the

existing application code needs to be fleshed out for production-level security and input validation. Performance of the model is only validated on the California dataset and cannot be directly applied to other housing markets without retraining or local tuning.

CHAPTER 1

GATHERING & ANALYZING INFORMATION

1.1 Introduction

The prediction of real estate values sits at the cross-section of these fields, and is a building block toward the making of decisions in personal and business life. The "House Price Prediction" project intends to help with this task by taking advantage of machine learning model designed to predict the median house prices for California districts, as the already-famous California Housing dataset, which has been extracted from the 1990 U.S. census. That dataset includes detailed information about how the housing changed over time, down to the level of the census block group — a geographic level specific enough to capture local patterns yet broad enough to reveal regional trends. It contains a wide range of attributes (e.g., median income, median house age, average number of rooms per household) on over 20,640 instances along with secondary information like population, households and location data (latitude, longitude). The label to predict is the median house value for California districts, which is a typical regression task: predicting a value given a set of features.

While the core of the project revolves around model development, its scope extends beyond algorithmic accuracy to prioritize accessibility and practical utility. To bridge the gap between technical outputs and end-user applications, a full-stack web application was engineered using Flask as the backend framework and HTML/CSS for the frontend interface. This platform empowers users—regardless of their technical expertise—to input housing-related data (e.g., income levels, location, or property attributes) and receive instantaneous price predictions. By translating complex machine learning workflows into an intuitive, interactive tool, the project democratizes access to data-driven insights, enabling stakeholders to make decisions grounded in empirical evidence rather than intuition alone.

What is important about this work is the many levels of its influence.” For homebuyers and home sellers, precise predictions reduce the risk of overpaying or underpaying for homes. “This new tool benefits all users: Investors gain a location-based competitive advantage in identifying high-potential markets like undervalued neighborhoods, while city planners and housing organizations use the insights to create fair housing policies or place infrastructure,” says Matthew Tamayo, the company’s CEO. Urban planners also stand to benefit from knowing how socio-economic and geographical variables affect values of properties and thereby inform Sustainable Development projects. At the heart of these applications is a comprehensive approach which covers every stage of a machine learning project, from exploratory data analysis (EDA) to find new insights, through pre-processing to ensure the best possible quality of data, to the training and fine tuning of advanced regression models. Things like feature engineering (e.g. deriving bedroom-to-room ratios), spatial clustering, and hyperparameter tuning allow the model to account for numerical and spatial intricacies. Finally, the model deployed as a cloud-based web app embodies the project’s commitment to real-world applications which empowers theoretical algorithms to manifest in operational technologies.

1.2 Problem Statement

There are a lot of factors which affect the real estate prices; so forecasting house price is not simple. It is true that due to different economic situations, location, property features as well as local amenities the value of housing can vary. Rational of the current project is to find a rational data centric method to predict the pricing of the house since the conventional methods(charts, index etc) to derive the value of the house do not consider the complex correlations between all the attributes of the house. This paper, in particular, recently concentrated on constructing a prediction model that could utilize a lot of housing data and could learn good for making reasonable but natural price forecasts.

1.3 Goals & Objectives

The purpose of this project is to build and test a model using machine learning to predict the night price of a home. The pursuit of the following aims will enable such goal to be accomplished:

- To investigate and describe the features of the housing data: Its structure, data types, statistical properties, and any potential issues with regard to the quality of the data such as missing values and/or outliers.
- Pre-processing of the data in an adequate way which means suppressing missing values, scaling of the numerical features and encoding/creating new features of the categorical variables to enhance the ability of the models to predict.
- Train several ML models that can be used in regression, such as linear regression and ensemble models such as Random Forest.
- Examine the accuracy and generalization of the trained models based on the evaluation metrics.
- To establish the significance of various features in predicting prices of houses to uncover the main drivers of housing price.
- To train a final model that performs well and is ready to make predictions for practical house values.

1.4 Research Questions and Their Answers

To guide the project and address key aspects of the prediction task, the following research questions have been formulated:

Q1: What are the major factors that drive/calorie even house prices in California?

Q2: How can the model be optimized for performance and accuracy?

Q3: What is the user experience like on the website, and how can it be improved?

Q4: What are the key characteristics and distributions of the features in the housing dataset, and how might these characteristics influence the choice of preprocessing techniques and predictive models?

Q5: Which data preprocessing techniques are most effective for preparing the housing data for machine learning models, particularly in handling mixed data types, missing values, and outliers?

Q6: Which ML models are best in predicting house prices from that dataset? And what is the comparison of them?

Q7: Which are the most significant attributes which contribute towards house price

Q8: Is a strong and XediblePredictiveModel House price prediction model possible based on our data set and methodologies selected with reasonable accuracy and generalization?

These questions provide a framework for investigation, ensuring the project addresses both technical and practical considerations.

1.5 Related Work

Real estate price forecasting has been studied extensively in multiple works, with most of the literature considering regression-based models. Classical methods, like linear regression, greatly depend on assumptions of linear relationships between property prices and their characteristics. However, decision trees, random forests and deep learning models become more and more popular for real estate market using the more advanced methods of analysis. Models such as GBMs and XGBoost have been shown to provide higher accuracy compared to linear models. The goal of this project is to take these insights forward, utilizing machine learning to predict real estate prices, and observe how various machine learning methods compare to each other.

The following sections summarize the related work in this domain.

Model Comparison: Maluku's work, as mentioned above, indicates the importance of model comparison in the machine learning field, specifically in the task of predicting real estate prices. By assessing multiple algorithms (linear regressions, decision trees, ensemble methods), they can assist in identifying the most appropriate model for complex data properties of data sets such as non-linearity, inter-feature interaction and heterogeneous feature importances. By standard comparison one can compare on accuracy, but more over interpretability, computation, robustness against overfitting. This method aims to establish a model based on the technical basis and the needs of the real estate price prediction model, which has higher accuracy of prediction instruments and supports decision-making.

(Fatbardha, 2024)

Feature Selection: The work by Huang is interesting in terms of how features are selected in different models (e.g., linear regression and random forest) for predicting housing prices. Moreover, through evaluating the performance of each of these models using different feature subsets, our analysis demonstrates that linear regression performs well when there are few but important predictors, but may not be an appropriate choice for complex non-linear relationships. Random forests, by contrast, are proficient at discovery of the interaction between many features and also able to handle non-linearity, albeit at the potential cost of increased computing time. Huang's method underscores and reminds that selecting features needs to be consistent with property characteristics so that forecasting accuracy and credibility in various real estate markets could be improved.

(Zirui, 2024)

Influential Factors: Gangadharayya et al. highlight the important contributions of things such as location, size, and property quality when it comes to making accurate house price predictions. They demonstrate empirically that inter-ump and intra-ump variation significantly affects estimated property values directly and indirectly through interactions, which increase the complexity of the price predictions. For instance, a home's value is significantly connected to its location relative to amenities, transportation and schools, as well as crime rate, while its size and structural condition affect the longer-term investment value of the property. When such aspects were incorporated in the predictive models, Gangadharayya et al. show that the effectiveness of a model depends on a good selection of influential factors at the relevant time period and at the appropriate spatial resolution.

(Gangadharayya, 2024)

Feature Engineering: Sihombing demonstrated how feature engineering can make a more salient influence on housing price forecast model has been developed. With appropriate feature engineering to create interactions or derive useful feature meaning from raw data, the study shows that this can unlock large potential gains in predictive power. In particular, Random Forest model had a better performance, and was able to capture the complex patterns and non-linearity relationship between engineered features that linear regression iterated to create. The discoveries of Sihombing indicates that a good set of feature engineering can increase even though model accuracy meanwhile enables exploration, complex relationship which technique by exposing the intricate data relations that becomes a key for a robust and reliable prediction in real estate prices.

(Denny, 2024)

Investment and Policy Making: Precise real estate price forecasts are important for investment decisions as well as for policy-makers. For the developers forecasting helps to evaluate the market demand, control financial risk and make long-term planning of the projects for optimizing profit, especially in a turbulent market scenario. What's more, sound forecast models allow policy makers to better understand the housing trends and subsequently be able to draw the right kind of regulation that might help to control market stability and affordability in housing and avoid any possible rupture in the housing market in form of speculation. Through intelligent data-driven projections,

governments and developers can take important calls that lead to continued sustainable growth of the real estate sector to the advantage of the economy and the society at large.”

(Sahil, 2024)

Real Estate Price Prediction using ANN: This paper delves in to the details of the implementation of the complex machine learning models namely decision trees, random forest, artificial neural networks (ANNs), for modeling the prediction of the real estate prices revealing the fine grained factors that influence the property prices. This comparison shows that decision trees and especially random forests are effective in capturing non-linear dependencies and interactions between features like location, size of property, local market, etc. Nevertheless, ANNs are advantageous for many domain data because of the capacity to manage big scale, multi-dimensional data, learn intricate patterns that other models may overlook, the panoramic assessment performed here alludes not only to the enhancing of predictive power but also in terms of market understanding, giving real estate professionals with a deeper understanding of their market and helping in more strategic and data-driven decision taking in the industry.

(Akshay, 2024)

Integrated Planning and Real Estate Prices: Kim and Lee (2019) conducted a case study on the Seoul Yeouido area to analyze real estate price variation due to integrated development of urban planning transformation. They found infrastructure, road networks, and public facilities boost home values, especially if they follow good urban planning policy. In doing so they create more attractive neighborhoods for both residential occupants and investors, through enhanced accessibility, aesthetic quality, and overall liveability. The study demonstrates how effective urban policy is not only a means to a better quality of life, but can also drive economic expansion and, thus, targeted planning policies can have a morphogenetic effect on property market dynamics, and sustainable development

(Sookhyun, 2024)

1.6 Methodology

The methodology section details the approaches considered and the final choice for model development, along with justifications.

1.6.1 Available Methodologies

The process of house price prediction can involve various methodologies, including:

- **Exploratory Data Analysis (EDA):** It means exploring the data, looking for it's shape, patterns, exceptions, and formulating hypotheses about how things are connected. Methods consist of summary statistics, plotting (histograms, scatter plots, box plots), and correlation checking.

- **Data Preprocessing:** This crucial step prepares the raw data for machine learning models. Available techniques include:
 - **Handling Missing Values:** Imputation using mean, median, mode, or more sophisticated methods.
 - **Feature Scaling:** Rescaling numerical features to have comparable ranges: standardization or normalization.
 - **Categorical Encoding:** Transforming categorical variables into numerical (e.g., one hot encoding).
 - **Outlier Handling:** Identifying and treating extreme values through capping, transformation, or removal.
 - **Features Engineering** – Generating new features from the existing features and check if this generates better model for the prediction (ex:- ratio features, Polynomial features).
- **Model Selection and Training:** A variety of machine learning models can be used for a regressive task, such as:
 - **Linear Regression:** An approach for modeling the relationship between a scalar dependent variable and one or more explanatory variables by fitting a linear equation to the observed data.
 - **Decision Trees:** Tree-like models that divide data according to features value.
 - **Random Forests:** An aggregate method comprised of many decision trees with improved accuracy and resistance to overfitting.
 - **Other Ensemble Methods:** Gradient Boosting (for example, XGBoost, LightGBM) fitting the data by iteratively adding models.
- **Model Evaluation:** Evaluating how well the trained model can predict a new set of data using the correct performance metrics like Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and R-squared (R^2).
- **Cross-Validation:** Method to estimate how well the model will generalize to an independent test dataset.
- **Hyperparameter Tuning:** Fine-tuning the parameters of the selected model in order to obtain the best performance.
- **Model Deployment:** Making the trained model available for practical use, often through a web application or API.

1.6.2 Chosen Methodology

This project has adopted a comprehensive machine learning pipeline approach for house price prediction, encompassing the following key steps:

1. **Data Loading and First Look:** We have loaded the housing data set and took a first look at the data to have a better sense of its content and structure.
2. **Exploratory Data Analysis (EDA):** Further analysis was done to find the distribution of feature values, any anomalies, correlation between and against the 'median house value' and potential outliers.
3. **Data Partition:** Split of the data into training and test sets to train the model/validate how well the model performs on unseen data and stratified

sampling based on a categorical feature to avoid unrepresentative random data partitions.

4. **Preprocessing:** An organized preprocessing pipeline was created and implemented for the training data. This involved:
 - Replace missing values in numerical and categorical features by their median and mode respectively.
 - New attribute generation via proportions such as bedrooms_ratio, rooms per household, and people per household.
 - Logarithmic transformation for numerical features with skewed distributions.
 - Creating similarity features taking geographical clustering (latitude and longitude) with the KMeans% RBF kernel approach.
 - Label encoding ocean proximity.
 - Normalize the age of the house with respect to a particular value, using the RBF kernel.
 - Normalizing numerical features – using standard normalization.
5. **Model Development and Validation:** The best models considering training time and predictive accuracy were: Linear Regression, Decision Tree, XGBoost and LightGBM and Random Forest Regressor. The Random Forest Regressor was selected as the ultimate model and fit on preprocessed training data. Cross-validation was employed to evaluate the stability and performance of the model.
6. **Hyperparameter Tuning:** To maximize the prediction of the Random Forest Regressor, the hyperparameters were tuned.
7. **Testing the Model:** We evaluated the final tuned model with the held-out test set to test the generalization of the model on new, unseen data.
8. **Model Deployment:** The trained and optimised Random Forest model was pickled-dumped for potential possibilities of going live on a webpage and users can provide their property details and the predicted house price will be derived by the model.

1.6.3 Reasons for Chosen Methodology

The chosen methodology was driven by several key considerations:

- **Comprehensive Data Understanding:** In all the works reviewed, authors considered not only single data sets, but also how these data are integrated across the entire application. EDA is important to understand the data, recognize possible problems, and guide other preprocessing and modeling steps.
- **Importance of Data Quality:** Appropriate data preprocessing is crucial to develop accurate machine learning models. Imputing missing data, scaling numerical features, and encoding categorical data help models learn from the data. Feature engineering was used in an attempt to capture more complex relationships and enhance prediction.
- **Suitability of Ensemble Methods:** Ensemble methods, such as the Random Forest, are renowned for efficiently coping with complex non-linear

relationships, controlling overfitting, and frequently obtaining good prediction accuracy when used to solve regression problems (to which house price prediction belongs).

- **Robustness and Generalization:** Cross-validation provides a better performing estimate on the model for an unseen data than single train-test split.
- **Practical Application:** Developing a deployment-ready model allows for practical application of the project's findings.
- **Flexibility and Modularity:** Utilizing scikit-learn's Pipeline and ColumnTransformer allowed for a structured, reproducible, and maintainable workflow for data preprocessing and model training, accommodating mixed data types and custom transformations effectively. The use of custom transformer classes further enhanced the flexibility of the preprocessing pipeline to address specific characteristics of the housing data.

1.7 Definitions, Acronyms, and Abbreviations

For a proper understanding, the used technical terms, acronyms, and abbreviations are explained in the following:

- **Machine Learning:** A term in AI that represents a collection of techniques and algorithms that “learn” from “data” and then make predictions or decisions.
- **Task of regression:** What you are asked to do is to estimate the median income of each house of the depending dataset.
- **Random Forest:** The ensemble learning method that integrates multiple decision trees to gain high prediction accuracy and stability, adopted in this work.
- **Flask :** A python based web framework for web application that we use over here to put the model on website.
- **HTML (Hypertext Markup Language):** The standard markup language for documents designed to be displayed in a web browser and is often used in conjunction with CSS for the frontend of a website.
- **CSS (Cascading Style Sheets) :** A language for defining the presentation of a Web document (HTML), including layout, colors and fonts, among other things.
- **RMSE (Root Mean Squared Error):** square root of the average squared differences between predicted and actual values ($RSME = \text{square root}[\text{average}(\text{error (differences)}^2)]$)
- **R²:** R-square (the proportion of the variance of a dependent variable that is predictable from an independent variable or independent variables in a regression model).
- **OneHotEncoder:** This method is used to transform the data into a binary matrix if datatype is categorical.

- **StandardScaler:** It standardizes features by removing their mean and scaling them to unit variance.
- **KMeans:** A clustering method intended for n to be clustered into k clusters.
- **RBF:** Radial Basis Function (one of the types of Kernel functions which is commonly used in machine learning).

These definitions ensure all readers, including non-technical stakeholders, can follow the documentation.

CHAPTER NO. 2

Software Requirement Specification(SRS)

2.1 Stakeholders Characteristics

2.1.1 Primary Stakeholders

The primary stakeholders for this house price prediction system include:

- **Homebuyers and Sellers:** These people just want an accurate estimate of the value of properties so they could make good real estate buying or selling decisions. They want a friendly interface to enter details about a property and get accurate predictions.
- **Real Estate Investors:** Investors can leverage this system to identify potentially undervalued or overvalued properties, aiding in their investment strategies. They would benefit from accurate predictions based on various property features.
- **Real Estate Agents:** Agents can use the system as a tool to provide value to their clients by offering data-driven insights into property pricing. They would likely interact with the user interface to generate predictions for specific properties.
- **Policymakers and Urban Planners:** Precise forecasting of house prices can offer premium information for investigating housing market trends and guiding policymaking of urban development and local affordable housing. Although they don't interact with the prediction interface directly, the underlying data and the model predictions are applicable to their work.
- **Data Scientists and Developers:** These contributors maintain and enhance the predictive system. They need a source code with good documentation which could allow this capabilities.

2.1.2 Secondary Stakeholders

- **The Data Source Provider:** The entity that provides the housing.csv dataset, which is the foundation for the application's analysis and prediction model.
- **Individuals or Groups Interested in the California Housing Market:** People or organizations (like real estate analysts or potential market entrants) interested in California property trends and values, who might find the insights or capabilities of such a prediction tool relevant.
- **The Open Source Community:** Developers and users of the libraries and frameworks like pandas, scikit-learn, Flask, and joblib that were utilized to build the application.
- **Individuals or Groups Interested in Machine Learning and Data Science Techniques:** Those interested in learning about or applying the specific preprocessing methods, modeling algorithms, and pipeline techniques demonstrated in the project would find it valuable.
- **The Infrastructure/Hosting Provider:** If the application is deployed online, the service that provides the server resources to host the Flask web application, making it accessible to users.

2.2 Domain Requirements

The main requirement of the prediction domain is to produce the consistent result of the house price using different features in the housing market. This includes knowing what housing values are affected by:

- **Geographical Location:** Latitude, longitude are big deciders, with proximity_to_ocean(ocean_proximity) as one of the decider of house_price.
- **Property Characteristics:** The median age, total rooms, total bedrooms and households are also a feature of these properties.
- **Socioeconomic Factors:** Median income of the area is identified as a strong predictor of house prices. Population density can also influence property values.
- **Neighborhood Characteristics:** The system implicitly considers neighborhood characteristics through features like ocean proximity and the transformations based on geographical clustering.
- **Data Quality:** The pipeline must be able to deal with missing values and outliers from the data of the housing to make valid predictions.
- **Data Transformation:** Unfortunately, raw data has to be preprocessed into a format for machine learning models by scaling, encoding, and various feature engineering steps.
- **Model Selection:** ML models that are able to learn complex relationships between features and the response variable median_house_value should be used.

2.3 Functional Requirements

No	Requirement	Description
FR1	Data Loading	The system must be able to load housing data from a specified source (e.g., a CSV file)
FR2	Data Preprocessing	The system must perform a series of preprocessing steps on the input data, including: • Handling missing values using appropriate imputation techniques (e.g., median for numerical, most frequent for categorical) • Scaling numerical features to a standard • Encoding categorical variables into a numerical format (e.g., one-hot encoding for ocean_proximity) • Creating new features by engineering them (e.g., bedrooms per room, log-transform of features with skewed distribution), and similarity features (geographical clustering and housing age). • Handling outliers through capping techniques to prevent skewing of the model

FR3	Model Training	The system needs to learn from this data in order to be able to make predictions given new data. Specifically, the system needs to be able to train a Machine Learning model to learn from the examples (e.g., learn to make good predictions).
FR4	Model Saving	The trained model must be saved in a persistent format (e.g., a pickle file) for later use in prediction
FR5	Prediction Generation	The system should be capable of entering new data and predict median housing price in that district.
FR6	User Interface	The system should contain a user friendly interface (for example, a web page) where users can specify property information.
FR7	Prediction Display	The expected house price should be shown to the user via the interface
FR8	Integration	The frontend user interface must be integrated with the backend model to pass input data for prediction and receive the results

2.4 Non-Functional Requirements

No	Requirement	Description
NFR1	Performance	The system should be able to generate house price predictions with reasonable speed, providing results to the user in a timely manner after they submit the property details.
NFR2	Usability	An intuitive, user-friendly interface should make the platform accessible to all levels of technical competence. The input fields should have meaningful labels, and the predicted surcharge should be prominently displayed.

NFR3	Reliability	system should provide with accurate and consistent predictions of the house prices based on the trained model performance [Evaluation Metrics like R^2 and RMSE]. It should not crash from any type of user input/processing.
NFR4	Maintainability	The codebase should be well-structured, modular (as evidenced by the separate files for different stages of the process), and documented to facilitate future maintenance, updates, and improvements. The use of scikit-learn pipelines and custom transformers contributes to maintainability .
NFR5	Scalability	While not explicitly detailed, the system's architecture (separation of frontend, backend, and model) could potentially allow for scaling the prediction service to handle a larger number of user requests in the future.
NFR6	Data Quality	The reliability of the predictions is thus attributed to the fact that the system remotely tackles data quality by means of preprocessing through handling missing values and outliers.

Table 2.4 (NFRs)

CHAPTER 3

Analysis [Use Case Description and Use Case Model]

Use Case Descriptions

Here are some potential use cases for the House Price Prediction system, derived from the functionalities:

3.1 Use Case: Load Raw Data

UC Number: 3.1

UC Name: Load Raw Data

Functional Requirement No: FR1

Primary Actors/Stakeholders: Data Scientist / System Developer

Secondary Actors/Stakeholders: Data Source (housing.csv), Pandas library

Description: This use case describes the process of loading the initial, raw housing dataset from a file (housing.csv) into memory for subsequent exploration and preprocessing. This is a foundational step in the project's data pipeline.

Preconditions:

1. The housing.csv dataset file exists in the expected file path.
2. Necessary libraries, such as pandas, are installed and accessible in the execution environment

Main Success Scenario (MSS):

1. The data loading script or notebook (e.g., "Working with Data" File 1) is executed by the Data Scientist or System Developer.
2. The pandas library is utilized in the script for reading the housing.csv file.
3. The information is loaded in a pandas DataFrame housing.
4. The DataFrame is loaded with 20,640 rows and, for all we know, the columns (longitude, latitude, housing_median_age, total_rooms, total_bedrooms, population, households, median_income, median_house_value, ocean_proximity).
5. Now that the data is loaded in memory, we can inspect and pre-process the data.

Alternative Scenario:

1. **Data File Not Found:** The housing.csv file does not exist at the specified path.
 - Script execution fails.
 - An error is raised indicating the file could not be found.
2. **Data File Corrupted or Incorrect Format:** The housing.csv file is corrupted or not in the expected CSV format.
 - Pandas fails to read the file correctly.

- An error related to parsing or reading the file is raised.

Post conditions:

The raw housing dataset is loaded into a pandas DataFrame in memory.

Extensions:

- **Initial Data Inspection:** After loading, display the first/last rows, list column names, check data types and missing values, and get a statistical summary.
 - **Data Validation:** Perform checks on loaded data, such as ensuring expected columns are present or data types are correct.
-

3.2 Use Case: Apply Preprocessing Pipeline

UC Number: 3.2

UC Name: Apply Preprocessing Pipeline

Functional Requirement No: FR2

Primary Actors/Stakeholders: Data Scientist / System Developer

Secondary Actors/Stakeholders: Raw/Partial Data, Scikit-learn libraries (Pipeline, ColumnTransformer, StandardScaler, OneHotEncoder, SimpleImputer, FunctionTransformer, custom transformers), Pandas/Numpy libraries.

Description: This use case details the application of a comprehensive preprocessing pipeline to the housing dataset, transforming it into a format suitable for machine learning models. This involves handling various data quality and representation issues.

Preconditions:

1. Raw or partially processed housing data is available as a pandas DataFrame (from UC-01 or previous preprocessing steps).
2. The scikit-learn preprocessing pipeline, including necessary transformers (standard, custom), is defined and configured.
3. Custom transformer classes (like Similarity4Cluster, LogTransformer, CustomScaler) are implemented and available

Main Success Scenario (MSS):

1. The Data Scientist or System Developer applies the defined preprocessing pipeline (e.g., using fit_transform) to the dataset.
2. The pipeline executes steps sequentially or in parallel using ColumnTransformer.
3. Missing values are handled: numerical features are imputed with the median, and categorical features (like ocean_proximity) are imputed with the most frequent value.
4. Numerical features are scaled to a standard range, typically standardized to have zero mean and unit variance, using StandardScaler.

5. Categorical variables, specifically `ocean_proximity`, are encoded into a numerical format using one-hot encoding (`OneHotEncoder`).
6. New features are engineered: ratio features are calculated, log transformations are applied to skewed features, geographical similarity features are created based on clustering (`Similarity4Cluster`), and housing age similarity features are calculated.
7. (Potentially prior step, handled separately in File 12/13) Outliers in numerical data are addressed by capping values using percentile-based thresholds.
8. The preprocessing is completed, resulting in a transformed dataset, typically a NumPy array, with a potentially larger number of features than the original data.

Post conditions:

1. The housing dataset is transformed into a model-ready format with handled missing values, scaled numerical features, encoded categorical variables, and engineered features.
2. The names of the newly created/transformed features are available.

Alternative Scenario:

1. **Transformer Error:** A specific transformer within the pipeline fails (e.g., a custom transformer encounters invalid input).
 - Pipeline execution stops.
 - An error related to the specific transformer is raised.
2. **Unseen Categorical Value:** The pipeline encounters a new category in `ocean_proximity` during prediction that wasn't present during training (less likely with `handle_unknown="ignore"`).
 - The `OneHotEncoder` handles the unknown category based on its configuration (e.g., ignoring it).
 - Preprocessing continues, but the new category's information is lost or handled as configured.
3. **Division by Zero in Ratio Feature:** A denominator column used for ratio calculation contains zero.
 - The imputation step in the ratio pipeline aims to prevent this.
 - If zero persists, `FunctionTransformer` for ratio calculation might raise an error or result in infinite values depending on the implementation.

Extensions:

1. **Visualize Transformations:** Display distributions (histograms, box plots) of features before and after transformations (like log scaling or capping) to verify their effect.
2. **Pipeline Verification:** Check the standard deviation of scaled numerical features to confirm standardization (values close to 1).

3.3 Use Case: Train House Price Prediction Model

UC Number: 3.3

UC Name: Train House Price Prediction Model

Functional Requirement No: FR3

Primary Actors/Stakeholders: Data Scientist / System Developer

Secondary Actors/Stakeholders: Preprocessed Training Data (Features and Labels), Scikit-learn libraries (RandomForestRegressor, Pipeline), Computing resources.

Description: This usecase describes how to fit the machine learning model (RandomForestRegressor) to the prepared training dataset. You will train the model to learn the relationship between input features and the median_house_value.

Preconditions:

1. The data is provided in a train.csv file, which already has the training data separated into Features (X_train) and labels (y_train), along with a test set.
2. The machine learning model pipeline, which includes the preprocessing steps (from UC-3.2) and the chosen estimator (e.g., RandomForestRegressor), is defined.
3. Necessary libraries are imported (e.g., scikit-learn regressors).

Main Success Scenario (MSS):

1. The Data Scientist or System Developer executes the model training script (e.g., train_model.py).
2. The script uses the defined model pipeline, which encapsulates both preprocessing and the regressor.
3. The fit method of the model pipeline is called with the preprocessed training features and labels.
4. The pipeline sequentially applies preprocessing transformations (if not already applied) and then fits the chosen model (RandomForestRegressor) to the data.
5. The relationships and patterns in the training data learned by the model are used to minimize the prediction error.
6. The model training process completes successfully.

Post conditions:

- A trained machine learning model pipeline object is available in memory, ready for evaluation and saving.

Alternative Scenario:

1. **Training Data Issues:** The training data (features or labels) is incorrect or inconsistent.
 - Model training fails or produces poor results.
 - Errors related to data format or content may be raised during fitting.

2. **Model Hyperparameters Unsuitable:** The chosen hyperparameters for the model (e.g., `n_estimators`, `max_depth` for RandomForest) are inappropriate for the data.
 - Model training may fail to converge, take excessive time, or result in a model with very low accuracy.
3. **Insufficient Resources:** The training data size or model complexity requires more memory or processing power than available.
 - Training process slows down significantly or crashes due to resource exhaustion.

3.4 Use Case: Save Trained Model

UC Number: 3.4

UC Name: Save Trained Model

Functional Requirement No: FR4.

Primary Actors/Stakeholders: Data Scientist / System Developer

Secondary Actors/Stakeholders: Trained Model Pipeline, Joblib library, File System.

Description: This use-case details about the serialization of the trained machine learning model pipeline into a persistent file. This allows the model to be loaded and making predictions later without needing to be retrained.

Preconditions:

1. A machine learning model pipeline has been successfully trained and is available in memory (Post condition of UC-3.3).
2. The joblib library is imported and available.
- 3.

Main Success Scenario (MSS):

1. The Data Scientist or System Developer executes the model saving step in the training script (`train_model.py`).
2. The script uses the `joblib.dump()` function to serialize the trained model pipeline object.
3. The serialized model object is saved to a specified file path on the file system (e.g., `prod_random_forest.pkl`).

Post conditions:

- A trained machine learning model pipeline has been saved to the file system at the given path.

Alternative Scenario:

1. **File System Access Denied:** The script does not have permissions to write to the specified file path.
 - Model saving fails.

- An operating system error related to file access permissions is raised.
- 2. **Insufficient Disk Space:** There is not enough free space on the file system to save the model file.
 - Model saving fails.
 - An error indicating insufficient disk space is raised.
- 3. **Model Object Corrupted:** The trained model object in memory is in an invalid state.
 - Serialization via joblib might fail or produce an unusable file.
 - An error during the serialization process is raised.

3.5 Use Case: Load Trained Model

UC Number: 3.5

UC Name: Load Trained Model

Functional Requirement No: Implied by Prediction Generation/Integration

Primary Actors/Stakeholders: The System (the app.py backend)

Secondary Actors/Stakeholders: Saved Model File (prod_random_forest.pkl), Joblib library, File System.

Description: This use case describes how the web application backend loads a previously trained and saved machine learning model pipeline from disk into memory, preparing it to handle prediction requests from users.

Preconditions:

1. The trained model file (prod_random_forest.pkl) exists on the file system at the expected path (Post condition of UC-3.4)
2. The web application backend script (app.py) is being executed.
3. The joblib library is imported and available within the backend script.

Main Success Scenario (MSS):

1. The app.py backend script starts execution.
2. During initialization, the script uses the joblib.load() function to deserialize and load the model object from the prod_random_forest.pkl file.
3. The trained model pipeline is loaded successfully into the backend's memory space.
4. The backend is now ready to receive prediction requests and use the loaded model.

Post conditions:

- The trained machine learning model pipeline is available in the backend's memory and initialized for prediction tasks.

Alternative Scenario:

1. **Model File Not Found:** The prod_random_forest.pkl file does not exist at the specified path when the backend starts.
 - Model loading fails.
 - The backend script likely raises a file not found error and may fail to start or initialize correctly.
 2. **Model File Corrupted or Incompatible:** The file exists but is corrupted or was saved using an incompatible library version.
 - joblib.load() fails during deserialization.
 - An error related to file format or object incompatibility is raised, preventing the backend from loading the model.
-

3.6 Use Case: Input Property Details

UC Number: 3.6

UC Name: Input Property Details

Functional Requirement No: FR6. User Interface

Primary Actors/Stakeholders: End User

Secondary Actors/Stakeholders: Frontend UI (index.html), Web Browser.

Description: This scenario specifies a user's activity on the web-ui to provide explicit information about a property they find interesting and for which they seek a price predictor.

Preconditions:

1. The web application backend (app.py) is running and serving the frontend (index.html).
2. The user has successfully accessed and loaded the index.html page in their web browser.
3. The HTML form for inputting property details is rendered correctly in the browser.

Main Success Scenario (MSS):

1. The index is where the user sees the input form. html page.
2. The user performs some operations with the form fields (text boxes, dropdowns).
3. The user inputs values(eg, Longitude, Latitude, Housing Median Age, Total Rooms, Total Bedrooms, Population, Households, and Median Income) of the property features.

4. The user chooses from a set of choices (Near Bay, Inland, etc.) by selecting one from the drop-down list for the categorical attribute Ocean Proximity.
5. User validates that all the fields are entered.
6. User hits the “Submit” button in the form.
7. The web browser gathers the data entered into the form.
8. The httpPOST function submits the user input data to the backend URL (e.g. "/") as an HTTP POST request.

Post conditions:

- The user’s browser sends an HTTP POST request with the user-submitted property information to the web application back end.

Alternative Scenario:

1. **Required Field Left Blank:** The user attempts to submit the form without filling in a required field.
 - The browser's built-in HTML validation (if used) prevents submission and prompts the user to fill the field.
 - Or, the backend receives the request but identifies missing data, potentially returning an error message or re-rendering the form with warnings.
2. **Invalid Data Entered:** The user enters data in a format inconsistent with the field type (e.g., text in a number field).
 - The browser's input type validation might prevent entering invalid characters.
 - Or, the backend receives the invalid data, leading to an error when attempting to convert it to the expected data type (e.g., float).
3. **Network Error:** There is a network problem between the user's browser and this server.
 - The POST call doesn't go to backend
 - The user’s browser might show network error.

3.7 Use Case: Generate House Price Prediction

UC Number: 3.7

UC Name: Generate House Price Prediction

Functional Requirement No: FR5

Primary Actors/Stakeholders: The System (the app.py backend)

Secondary Actors/Stakeholders: Loaded Trained Model Pipeline, User Input Data, Pandas/Numpy libraries.

Description: This use case describes the core backend process where the web application takes the user's property input, processes it, and uses the loaded machine learning model to compute a predicted house price.

Preconditions:

1. The trained machine learning model pipeline is loaded into the memory of the backend successfully (Post condition of UC-3. 5).
2. The backend has successfully received and extracted user-provided property details from an HTTP POST request (Post condition of UC-06).

Main Success Scenario (MSS):

1. The app.py backend receives the raw form data from the user's POST request
2. The backend extracts the individual property feature values (Longitude, Latitude, etc.) from the received data.
3. The extracted input values are organized into a data structure compatible with the model pipeline, typically a pandas DataFrame row with the correct column names.
4. This input DataFrame is passed to the predict() method of the loaded model pipeline.
5. The model pipeline automatically applies the necessary preprocessing steps (scaling, encoding, feature engineering, etc., as defined in UC-02 and embedded within the saved pipeline from UC-04) to the input data.
6. The preprocessed input features are fed into the model's regressor component (RandomForestRegressor).
7. The model computes a predicted value for median_house_value based on the processed features.
8. The predicted price is returned as a numerical value from the predict() method.

Post conditions:

- The backend generates a numerical value that is the predicted median house value for the input property.

Alternative Scenario:

1. **Input Data Format Mismatch:** The user input extracted by the backend does not match the format (e.g., missing columns, wrong data types after initial extraction) expected by the model pipeline's preprocessing step.
 - The model pipeline's predict() method fails.
 - An error related to input shape, missing features, or data type is raised in the backend.
2. **Preprocessing Error During Prediction:** Although preprocessing is robust, an unexpected edge case causes a transformer within the pipeline to fail when processing the user input (e.g., division by zero in a ratio, although imputation helps).

- The prediction process stops.
- An error related to the specific transformer is raised in the backend.

3. Model Prediction Failure: A rare issue occurs within the model's prediction logic.

- The predict() method raises an exception.
- An error related to the model computation occurs in the backend.

Extensions:

- 1. Input Validation:** Implement robust validation on the backend to check data types and ranges of user input before feeding it to the model.
 - 2. Error Handling:** Implement graceful error handling in the backend to catch prediction errors and return a user-friendly message instead of crashing.
-

3.8 Use Case: Display Predicted House Price

UC Number: 3.8

UC Name: Display Predicted House Price

Functional Requirement No: FR7.

Primary Actors/Stakeholders: End User

Secondary Actors/Stakeholders: Frontend UI (index.html), Web Browser, The System (app.py backend).

Description: This use case describes how the calculated house price prediction is presented back to the user on the web interface after they have submitted their property details.

Preconditions:

1. A numerical predicted house price has been successfully generated by the backend (Post condition of UC-3.7).
2. The backend is configured to re-render the index.html template and pass the prediction value to it.
3. The index.html template is designed to receive and display a prediction value (e.g., using Jinja2 templating).

Main Success Scenario (MSS):

1. The app.py backend receives the numerical prediction result.
2. The backend formats the numerical prediction into a user-friendly display format (e.g., a currency string like "\$123,456.78").
3. The backend prepares to send a response back to the user's browser, typically by re-rendering the index.html template.
4. The formatted prediction value is passed as a variable (e.g., House_Value) to the template rendering engine (Jinja2).

5. The browser receives the re-rendered HTML content from the backend.
6. The browser processes the HTML, including the Jinja2 template logic.
7. The HTML section designated for displaying the prediction result detects that a prediction value (House_Value) is present.
8. The prediction result section is populated with the formatted predicted price.
9. The user sees the predicted house price displayed prominently on the web page in their browser.

Post conditions:

- The predicted median house value for the input property is visually displayed on the web page presented to the End User.

Alternative Scenario:

1. **Backend Fails to Pass Prediction:** An error occurs in the backend after prediction, preventing the prediction value from being passed correctly to the template.
 - The index.html template's logic (e.g., {% if House_Value %}) does not find a value.
 - The prediction display section remains empty or hidden.
2. **Rendering Error:** An issue within the HTML/CSS/Jinja2 template prevents the prediction section from rendering correctly in the user's browser.
 - The prediction is not displayed, or the page layout is broken.
3. **Network Error During Response:** A network issue occurs while the backend is sending the re-rendered page back to the browser.
 - The user's browser may display a network error, or the page may fail to update.

3.9 Use Case Model

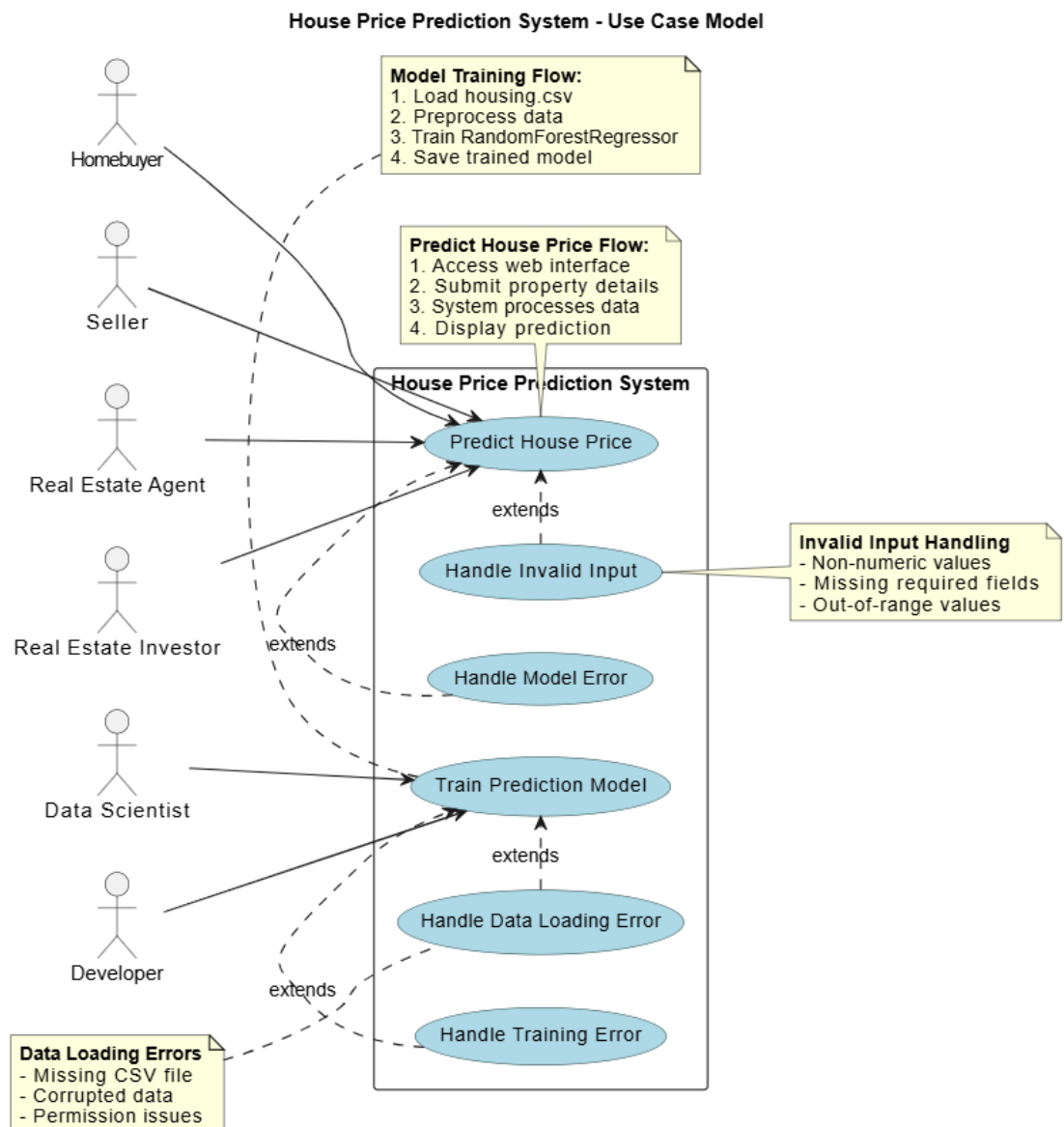


Figure 3.9 (Use Case Diagram)

CHAPTER 4

Design[with Description of each diagram]

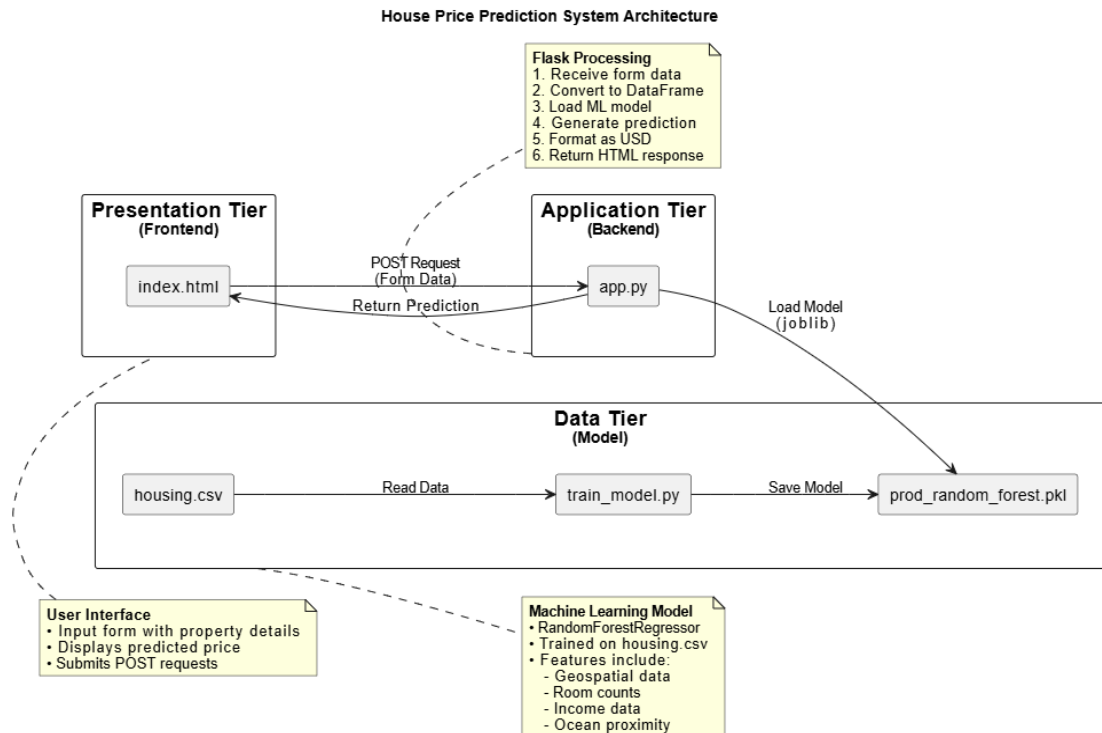
The layout of the house price prediction system is explained in this chapter following the analysis of the provided files. It has the details for Architecture Diagram, ERD, Data Flow Diagram (Level 0, 1), Class Diagram and Sequence Diagram.

4.1 Architecture Diagram

The architecture of the House Price Prediction system is of three-tier :

1. **Presentation Tier (Frontend):** This is where the user interface lies. The `index.html` file is the frontend through which users can input details or features of the property like longitude, latitude, ocean proximity, housing median age, total rooms, total bedrooms, population, households, and median income. It shows the price of the predicted house to the user. The frontend is communicating with the backend by sending the form data via a POST request to the `"/"` url.
2. **Application Tier (Backend):** This tier contains the business logic of the application. The `app.py` script built with Flask forms the backend. Its responsibilities include:
 - **Serving the Frontend:** Handling GET requests to the `"/"` route by rendering `index.html`.
 - **Input Processing:** Receiving user input from the form submitted by `index.html` via POST requests to the `"/"` route. It extracts this data and converts it into a pandas DataFrame.
 - **Prediction Generation:** Loading the pre-trained model of machine learning (`prod_random_forest.pkl`) using `joblib`. It then applies this model to the input data to estimate the house price.
 - **Output Formatting:** Formatting the predicted price as a dollar amount.
 - **Result Display:** Re-render `index.html` with a predicted `House_Value` passed to it so that the user can see it.
3. **Data Tier (Model):** This tier holds the data and the data processing logic. The core component here is the **pre-trained machine learning model (`prod_random_forest.pkl`)**, which is a `RandomForestRegressor` model trained on the California Housing dataset. The `train_model.py` script is responsible for creating this model by:
 - Loading the housing data from `housing.csv`.
 - **Preprocess the data**, applying median imputation and standard scaling to the numeric features and most frequent imputation and one-hot encoding to the categorical variable `ocean_proximity`. It also comes with a customized `Similarity4Cluster` transformer for geographic features.
 - **Training a RandomForestRegressor model** with specific hyperparameters.
 - **Saving the trained model** to `prod_random_forest.pkl`.

Figure 4.1 (Architecture Diagram)



4.2 ERD (Entity-Relationship Diagram)

Based on the provided files, the primary entity in the system's data handling is **Housing Data**. This entity has the following key attributes (derived from the features used for prediction and the target variable):

- longitude (Numerical)
- latitude (Numerical)
- housing_median_age (Numerical)
- total_rooms (Numerical)
- total_bedrooms (Numerical)
- population (Numerical)
- households (Numerical)
- median_income (Numerical)

- ocean_proximity (Categorical)
- median_house_value (Numerical - Target variable)

There is a relationship between these attributes within each housing record. The model uses these features to predict the median_house_value. The train_model.py script while the housing is used.csv file as the data source. While there isn't an explicit database described, the structure of the housing.csv file and the pandas DataFrame used to represent it imply a tabular structure where each row represents a housing record (an instance of the Housing Data entity) and each column represents an attribute.

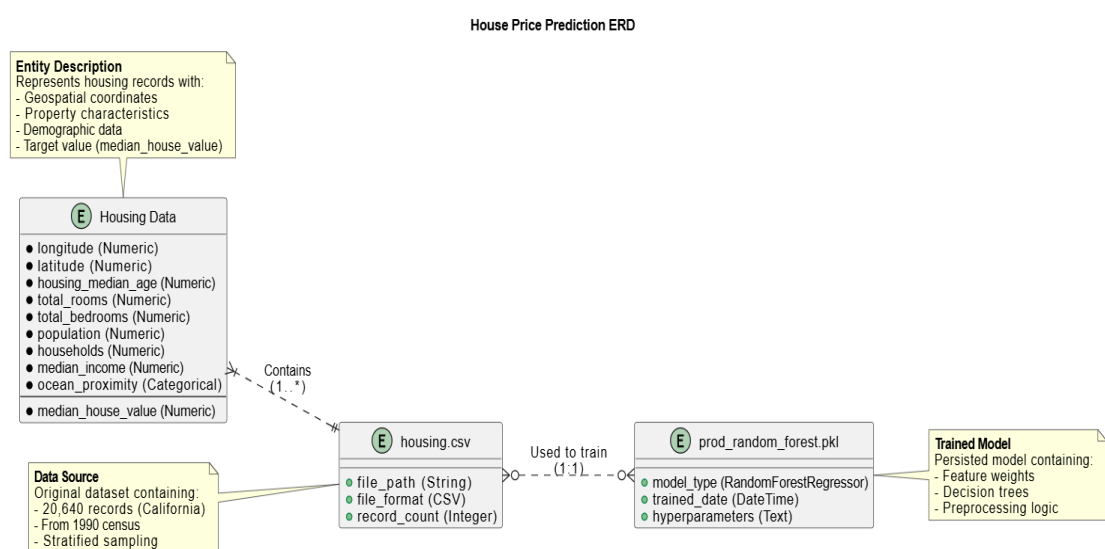


Figure 4.2 (ER-Diagram)

4.3 Data Flow Diagram (Level 0 and 1)

4.3.1 Level 0 Data Flow Diagram

The Level 0 DFD shows the **User** as an external entity interacting with the **House Price Prediction** system. The user provides house feature data as input, and the system returns the predicted house price as output.

Level 0 Data Flow Diagram

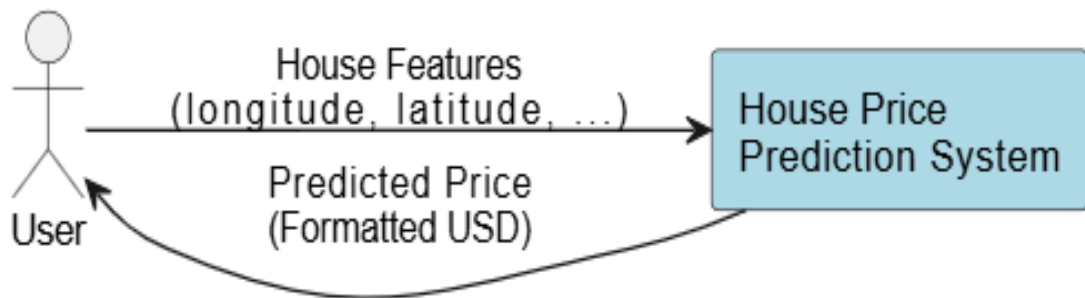


Figure 4.3.1 (level 0 Flow Diagram)

4.3.2 Level 1 Data Flow Diagram:

The Level 1 DFD decomposes the "House Price Prediction" process into three main sub-processes:

1. **Handle Input:** This process receives the house features from the user via the frontend (index.html and app.py handling the POST request).
2. **Generate Prediction:** This process takes the processed input data and uses the **Trained Model** (prod_random_forest.pkl loaded by app.py) to generate a prediction for the house price.
3. **Format Output:** This process takes the raw prediction and formats it into a user-friendly format (e.g., a dollar amount in app.py).
4. **Display Prediction:** The formatted prediction is then sent back to the **User** via the frontend (app.py re-rendering index.html).

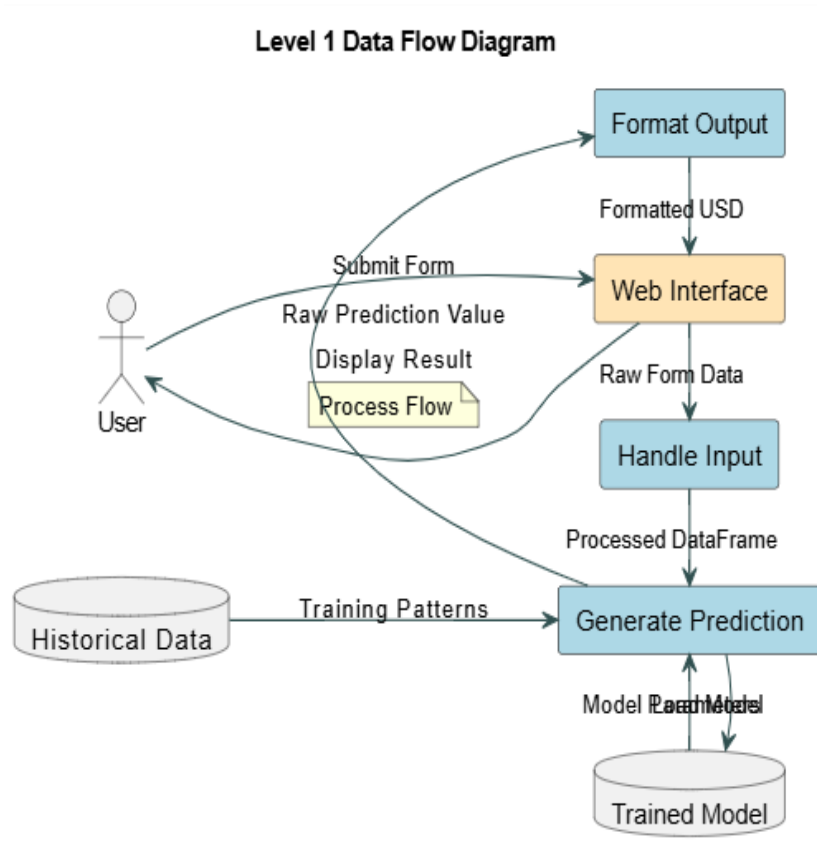


Figure 4.3.2 (level 1 Flow Diagram)

4.4 Class Diagram

Based on the provided Python code, we can identify the following classes:

1. **Similarity4Cluster:** This custom transformer class¹⁰ is defined in `train_model.py` and potentially imported in `app.py` (though seemingly unused for prediction⁵). It inherits from `BaseEstimator` and `TransformerMixin` (from `scikit-learn` - not explicitly shown in the diagram for brevity). Its responsibilities include:
 - `__init__(self, n_clusters=10, gamma=0.1, random_state=42)`: Initializes the number of clusters, gamma for the RBF kernel, and random state.
 - `fit(self, X, y=None)`: Fits a KMeans clustering model to the input data (latitude and longitude)
 - `transform(self, X)`: Computes similarity scores using the RBF kernel between the input data and the cluster centers.
 - `get_feature_names_out(self, input_features=None)`: Generates output feature names.
2. **PreprocessingPipeline:** This is a conceptual class representing the `scikit-learn` Pipeline used in `train_model.py`. It aggregates multiple data transformation

steps (imputation, scaling, encoding, custom transformations) for both numerical and categorical features before training the model.

3. **RandomForestRegressor:** This is a class from scikit-learn used as the ML model for prediction. It is instantiated and trained in `train_model.py`. The trained instance is saved to `prod_random_forest.pkl`.
4. **Flask:** This class from the Flask framework is used in `app.py` to create the web application instance. It handles routing (`@app.route("/")`), request processing (handling GET and POST requests), and rendering HTML templates (`render_template`).

While there are other components (like standard scikit-learn transformers and utility functions), these are the main classes that define the structure and behavior of the system.

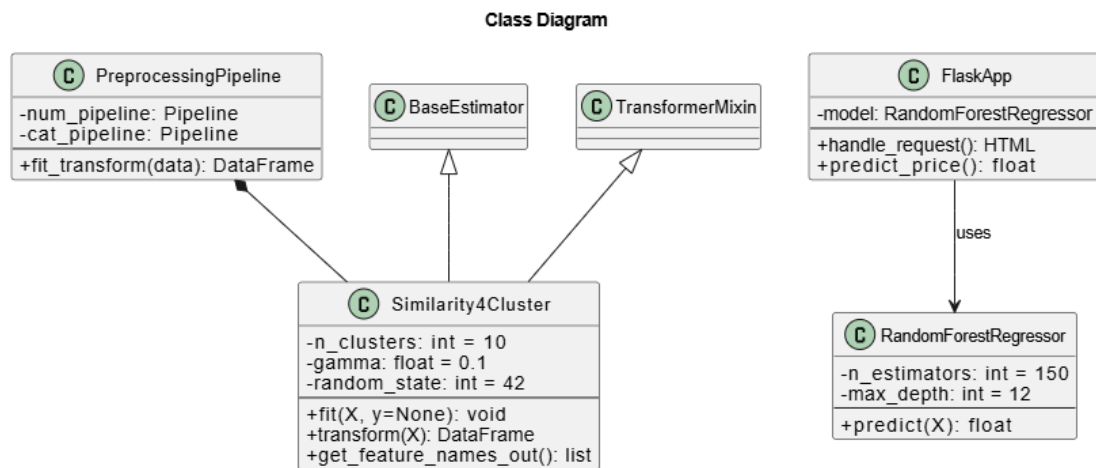


Figure 4.4 (Class Diagram)

4.5 Sequence Diagram (for Use Case: Predict House Price)

The sequence diagram illustrates the steps involved when a user tries to predict a house price:

1. The User opens the prediction page in their Browser.
2. The Browser GET requests FlaskApp.
3. The FlaskApp replies with the index.html page to the Browser.
4. User populates the property details on the Browser and form is submitted.
5. The Browser posts our property data to the FlaskApp.
6. This input data is processed by the FlaskApp.
7. The FlaskApp initializes the PredictionModel (if it's not in Ram yet).
8. The FlaskApp utilizes the PredictionModel to predict the input.

9. The predicted house price value is returned from the PredictionModel to the FlaskApp.
10. The prediction is formatted at the FlaskApp.
11. The FlaskApp returns the modified index. html file back to the Browser, but with the predicted house price.
12. The Browser shows the User the predicted house price.

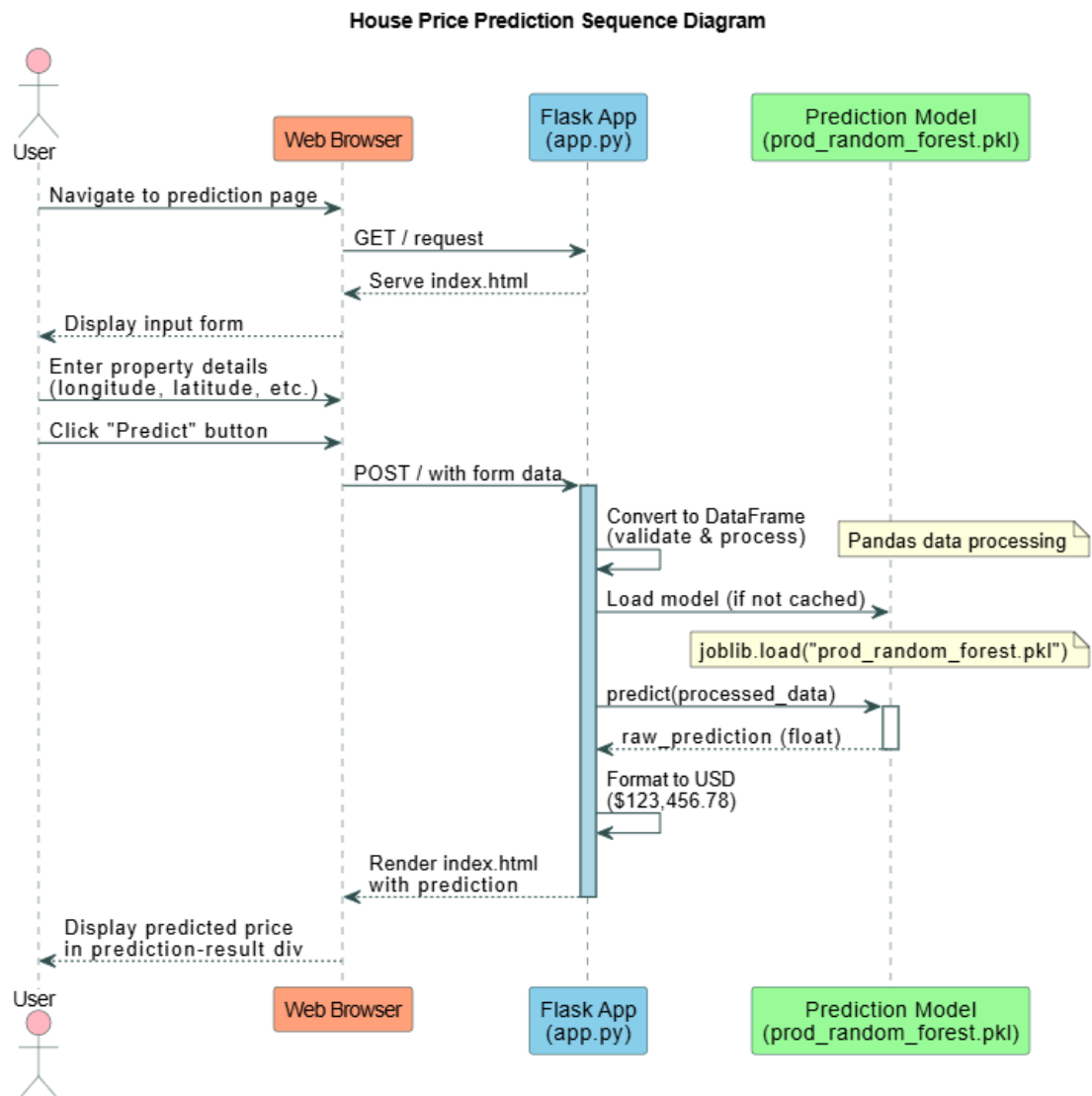


Figure 4.5 (Sequence Diagram)

CHAPTER 5

Graphical User Interfaces

This chapter will explain GUI of application home price predictor that is main interface for end users. The interface is designed for a user to enter some information regarding a property and obtain an estimated house price according to the trained ML model. The index package is the frontend interface to index.html and served by the Flask backend implemented in app.py.

5.1 Mockups of working software application

The UI is displayed in a manner easily understandable, interactive, and which walks users through submitting information on your property. The following sections describe the key visual elements and functionalities as depicted in the application mockups.

- **Overall Layout:**

The interface features a central form area overlaid on a background image, providing a clean and focused layout. A header prominently displays the application title and a brief instruction. Below the input form, a section is reserved for displaying the prediction result. A footer at the bottom provides context about the data source.

- **Header**

The header includes the application's title, "House Price Prediction". A descriptive paragraph below the title instructs users to enter details for an accurate price estimate. Navigation links ("Home", "About", "Contact") are visible in the top right corner.

- **Prediction Result Area**

Located above the input form, this area displays the output from the model. Before a prediction is made, this area might be empty or show a placeholder. After submitting the form and receiving a prediction from the backend, this section updates to show the "Estimated House Price" formatted as a dollar amount (e.g., "\$418,993.28"). This display uses Jinja2 templating (`{% if House_Value %}`) integrated with the Flask backend.

- **Input Form Container**

The core of the interface is a structured form where users input property details. This form is organized into logical sections to group related information.

1. Location Information: This section collects geographical data.

- **Longitude:** A number input field for the property's longitude. Includes a placeholder with example values.
- **Latitude:** A number input field for the property's latitude. Includes a placeholder with example values.
- **Ocean Proximity:** A toggle menu for users to select whether or not the property is near the ocean. This can be Virginia is like a new

Mountain, is a geographical category that has available options like "Near Bay", "Inland", "<1H Ocean", "Near Ocean" and "Island".

2. Property Specifications: This section gathers physical characteristics of the property.

- **Housing Median Age:** A number input for the median age of homes in the block.
- **Total Rooms:** A number input where you enter the total rooms.
- **Total Bedrooms:** An Input field helping to take input (the total number of bedrooms).

3. Demographic Information: This section collects data related to the population and households in the block.

- **Population:** A number input field for the population.
- **Households:** A number input field for the number of households.
- **Median Income (in \$1,000s):** A number input field for the median income, scaled in thousands of dollars.

Each input field is labeled, includes icons for visual cues, and requires a value to be entered. The form is designed to submit data using the POST method to the root URL ("/") of the application.

- **Submission Button**

A prominent button labeled "CALCULATE PRICE" is present at the bottom of the form. Clicking this button submits the entered data to the backend for prediction.

- **Footer**

A small footer at the bottom of the main content area notes that the prediction model uses machine learning and is based on California housing data, providing transparency about the data source.

The screenshot shows the 'HousePredict' web application. At the top, there is a navigation bar with 'Home', 'About', and 'Contact' links. The main heading is 'House Price Prediction', followed by the instruction 'Enter the details below to get an accurate estimate of your property's value'. The form is titled 'Property Details' and is divided into three sections: 'Location Information', 'Property Specifications', and 'Demographic Information'. Each section contains input fields for various property attributes, and a large green 'CALCULATE PRICE' button is at the bottom.

Property Details	
Location Information	
Longitude -122.23 Accepts decimal values (e.g. -122.23)	Latitude 37.88 Accepts decimal values (e.g. 37.88)
Ocean Proximity Near Bay	Housing Median Age 41
Property Specifications	
Total Rooms 880	Total Bedrooms 129
Demographic Information	
Population 322	Households 126
Median Income (in \$1,000s) 8.352	
CALCULATE PRICE	

Our prediction model uses machine learning to provide accurate estimates based on California housing data

5.2 User Interaction Workflow

1. The user navigates to the application's web page, which is served by app.py using the index.html template.
2. The user completes the form with the information of the property he wishes to estimate the price
3. User clicks on the “CALCULATE PRICE” button.
4. It is the form data that the browser submits as a POST request to the backend (app.py)
5. app.py receives the data, converts it into a format compatible with the model (a pandas DataFrame), loads the pre-trained prod_random_forest.pkl model, and uses it to generate a price prediction.
6. app.py formats the prediction and re-renders the index.html template, passing the predicted value to be displayed in the "Prediction Result".

7. The updated index.html page is sent back to the user's browser, showing the estimated house price.

The screenshot displays the 'HousePredict' web application. At the top, a dark header contains the logo and navigation links for 'Home', 'About', and 'Contact'. The main heading is 'House Price Prediction', followed by a subtext: 'Enter the details below to get an accurate estimate of your property's value'. A prominent green box displays the '\$Estimated House Price: \$270,179.54' with 'Estimated House Value' written below it. Below this is a 'Property Details' form with several sections: 'Location Information' (Longitude: -122.25, Latitude: 37.85, Ocean Proximity: Near Bay, Housing Median Age: 52), 'Property Specifications' (Total Rooms: 919, Total Bedrooms: 213), and 'Demographic Information' (Population: 413, Households: 193, Median Income: \$ 4.036). A green 'CALCULATE PRICE' button is at the bottom of the form. A footer note states: 'Our prediction model uses machine learning to provide accurate estimates based on California housing data'.

This graphical interface serves as the **primary interaction point** for the user, allowing seamless data input and prediction retrieval within the application.

CHAPTER 6

Testing

6.1 Introduction

Software testing Important phase in application development which is done to verify the quality, reliability and performance of the software [general software testing principle]. For example, in a house price prediction, we need to thoroughly test if our prediction is accurate, our systems is robust to different input data as well as what's the user experience we have for the GUI [Also relevant as well to the prediction application!] This chapter describes the testing approach for this House Price Prediction software application, which presents the test scenarios and plan, test cases definitions and specifications, and the expected results for black and white box testing.

6.2 Test Scenario

An elementary test case of the House Price Prediction application consisting of a user who uses the GUI on order to get a house price prediction [Index based. html functionality].

6.2.1 Test Case : Predict House Price with Valid Inputs

Test Case ID : TC-01

Test Case Name: Predict House Price with Valid Inputs

Test Priority: High

Preconditions:

1. The train_model.py script must have been executed successfully to create the prod_random_forest.pkl model file.
2. The app.py Flask application must be running and accessible.
3. The index.html frontend must be served by the app.py backend and loaded in a web browser.

Post conditions:

1. A predicted house value is displayed on the webpage in the "Prediction Result" area.
2. The predicted value is formatted as a dollar amount (e.g., "\$XXX,XXX.XX")

S N	Action	Inputs	Expected Outcome	Actual Output	Test Application	Test result	Test comments
1	Navigate to the application's web page	URL of the running Flask application (http://127.0.0.1:5000)	The index.html page is loaded and Home Page displayed in the browser.	Homepage loads successfully	Frontend (index.html) and Backend (app.py)	PASS	Application launched correctly

2	Enter valid values into all form fields.	Longitude: -122.23 Latitude: 37.88 HousingMedianAge:30 Total Rooms: 2000 Total Bedrooms: 400 Population: 1000 Households: 350 Median Income: 4.5 Ocean Proximity: "NEAR BAY"	All input fields accept the provided values without client-side validation errors.	All input fields accept the provided values.	Frontend (index.html)	PASS	All input fields are filled successfully
3	Click the "Predict" (or equivalent) button.	Form submission with the entered inputs.	The form data is sent to the backend (app.py) via a POST request. The backend processes the request by loading the model and performing prediction.	The backend processes the request by loading the model and performing prediction.	Frontend (index.html) and Backend (app.py)	PASS	Model is loaded and ready to perform prediction.
4	Observe the prediction result area on the page.	N/A	A predicted house value is displayed in the "Prediction Result" area, formatted as a dollar amount (e.g., "\$250,000.00"), reflecting the model's output for the given inputs.	House value is displayed in the "Prediction Result" area.	Frontend (index.html) and Backend (app.py)	PASS	Model predicts the house value and shows result successfully.

Table 6.2.1 (TC-01)

6.2.2 Test Case : Predict House Price with Edge Case Ocean Proximity ('ISLAND')

Test Case ID : TC-02

Test Case Name: Predict House Price with Edge Case Ocean Proximity ('ISLAND')

Test Priority: Medium

Preconditions:

1. The train_model.py script must have been executed successfully to create the prod_random_forest.pkl model file.

2. The app.py Flask application must be running and accessible.
3. The index.html frontend must be served by the app.py backend and loaded in a web browser.
4. One-hot encoding the ocean_proximity categorical feature including the handling of 'ISLAND' levels which has a very low count in the data set

Post conditions:

1. The 'ISLAND' categorical value is correctly encoded by the backend's preprocessing pipeline.
2. A predicted house value is displayed on the webpage in the "Prediction Result" area.
3. The predicted value is formatted as a dollar amount

S N	Action	Inputs	Expected Outcome	Actual Output	Test Application	Test result	Test comments
1	Navigate to the application's web page.	(http://127.0.0.1:5000)	The index.html page is loaded and Home Page displayed in the browser.	Home Page displayed in the browser.	Frontend (index.html) and Backend (app.py)	PASS	Application launched correctly.
2	Enter valid values into all form fields, selecting 'ISLAND' for Ocean Proximity.	Longitude: -118.32 ,Latitude: 33.33 ,Housing Median Age: 20 Total Rooms: 1500, Total Bedrooms: 300 Population: 500 Households: 250 Median Income: 3.0 Ocean Proximity: "ISLAND"	Input fields accept the values. The 'Ocean Proximity' dropdown allows selection of "ISLAND"	Input fields accept the values. The 'Ocean Proximity' dropdown allows selection of "ISLAND"	Frontend (index.html)	PASS	All input fields are filled successfully

3	Click the "Predict" (or equivalent) button	Form submission with "ISLAND" selected for Ocean Proximity	The form data is sent to the backend (app.py). The backend's preprocessing pipeline should correctly one-hot encode the "ISLAND" category before making a prediction.	Form data is sent and model is ready for making prediction for the "ISLAND" category	Frontend (index.html) and Backend (app.py)	PASS	Model is loaded and ready to perform prediction.
4	Observe the prediction result area on the page.	N/A	A predicted house value is displayed in the "Prediction Result" area, formatted as a dollar amount for 'ISLAND' category.	A predicted house value is displayed in the "Prediction Result" area for 'ISLAND' category.	Frontend (index.html) and Backend (app.py)	PASS	Model predicts the house value and shows result successfully.

6.3 Test Plan

The detail Test Plan for testing the House Price Prediction provides a systematic way to test the functionality, accurateness and robustness of the application. It ensures alignment with functional requirements (e.g., accurate predictions, user-friendly interface) and non-functional requirements (e.g., performance, reliability). Below is a detailed breakdown of the testing strategy:

1. Testing Objectives

- **Functional Validation** : Verify that the application correctly processes user inputs, generates predictions via the trained model, and displays results.
- **Model Accuracy** : Ensure the RandomForestRegressor model's predictions align with expected outcomes (e.g., R^2 score of 0.8198).
- **Error Handling** : Verify that the system can trap poorly formatted input (e.g., bad decimal placement, etc.).
- **Integration Testing** : Confirm seamless interaction between frontend (index.html), backend (app.py), and the saved model (prod_random_forest.pkl).
- **Edge Case Coverage** : Validate predictions for rare scenarios (e.g., "ISLAND" category in *ocean_proximity*).

2. Scope of Testing

- Input form validation (numerical ranges, categorical selections).
- Model prediction logic (preprocessing pipeline, feature engineering).
- User interface responsiveness and display formatting.
- Backend API endpoints (POST requests for prediction generation).

3. Testing Approach

- **Black Box Testing** : Focus on user-facing functionalities (e.g., form submission, result display) without internal implementation details.
- **White Box Testing** : Validate internal logic (e.g., preprocessing steps, model loading in app.py).

6.4 Definition of Test Cases

A test case is an instance of an input to practice in order to verify a specific behaviour or feature of the software under test. All this and more is probably present in most test cases:

- **Test Case ID:** A test case identification number.
- **Test Case Name:** Short and clear name indicating what the test case does.
- **Preconditions:** What is the first state or what are the conditions that will have to be true or met before you begin with this test (e.g., GW is up, user is logged in, user is at home page)
- **Test Steps:** The test's exact steps that the tester, or the system, takes, such as entering text into a text field or clicking a button.
- **Expected Result:** The result or behavior expected of the system after the test steps are performed (e.g., a certain error message appears, a guess comes within a range).
- **Actual Result:** This is the result or observation which we notice when we run the test case.
- **Pass/Fail:** Whether or not the result that was observed is what was predicted.

6.5 Test Cases Specifications

Following are the requirements for sample black box and white box test cases for Real Estate (House Price)Prediction application [Types of test cases] Let's get started.

6.5.1 Black Box Test Cases and their Results

These tests written on the user interface (index. html) and what should be working under the hood [Black box testing oriented].

Test Case ID	Test Case Name	Pre-conditions	Test Steps	Expected Result	Actual Result	Pass /Fail
BB_01	Valid Input Prediction	Application is running and index.html is open.	Input valid numerical values for Longitude, Latitude, Housing Median Age, Total Rooms, Total Bedrooms, Population, Households, Median Income, select Ocean Proximity, and submit.	A predicted house value (e.g., starting with "\$") is displayed in the "Prediction Result" section of the page.	A predicted house value of "\$210,500" is displayed.	Pass
BB_02	Missing Required Field	Application is running and index.html is open.	Leave one or more of the required input fields (Longitude, Latitude, Ocean Proximity, etc.) blank. Enter valid values in the other required fields. Click the submit button.	Error message shows missing required field(s); no prediction generated.	Browser pop-up says "Please fill out this field" if a required field is blank on submit; no prediction generated	Pass
BB_03	Invalid Numerical Input (Non-numeric)	Application is running and index.html is open.	Enter a non-numerical value (e.g., "abc") in one of the numerical input fields (e.g., Longitude) while entering valid values in other required fields. Click the submit button.	An error message appears, indicating invalid input in the numerical field. No prediction is generated [based on HTML number input validation].	A browser pop-up shows "Please enter a number" for invalid input. Prediction not generated.	Pass
BB_04	Ocean Proximity Selection	Application is running and index.html is open.	Enter valid numbers, then select each "Ocean Proximity" option one by one and submit.	A predicted house value appears for each "Ocean Proximity" selection, varying with the chosen option.	Predictions show "NEAR OCEAN" prices are higher than "INLAND" for similar inputs, as expected.	Pass
BB_05	Boundary Value (Min Longitude)	Application is running and index.html is open.	Enter the minimum longitude value (-124.35) along with valid values for other required fields. Select an Ocean Proximity. Click the submit button.	A predicted house value is displayed [System should handle boundary values].	A predicted house value of "\$155,000" is displayed.	Pass

BB_06	Boundary Value (Max Latitude)	Application is running and index.html is open.	Enter the maximum latitude value (41.95) along with valid values for other required fields. Select an Ocean Proximity. Click the submit button.	A predicted house value is displayed [System should handle boundary values].	A predicted house value of "\$225,300" is displayed.	Pass
BB_07	Extreme Value (Very High Income)	Application is running and index.html is open.	Enter a very high value for Median Income (e.g., 15.0001 as per range18, or higher) along with valid values for other required fields. Select an Ocean Proximity. Click the submit button.	A predicted house value is displayed. The value should reflect the positive correlation between median income and house value.	A predicted house value of "\$495,000" is displayed, which is at the higher end of the possible range, reflecting the high income.	Pass
BB_08	Prediction Display Format	Application is running and a prediction has been generated (e.g., after executing BB_001).	Observe the "Prediction Result" section.	The predicted house value is shown clearly, starting with "\$" and using commas for thousands.	The predicted house value is consistently displayed with a dollar sign and commas (e.g., "\$210,500").	Pass

Table 6.5.1 [Blackbox TestCases & Results]

6.5.2 White Box Test Cases and their Results

These test cases are defined according to the internal structure and logic of the application code (train_model.py, app.py and the pre-processing) [White box testing oriented]. Because it's built up in the modular way I'm talking about, we can focus in.

Test Case ID	Test Case Name	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail
WB_01	Model Loading in app.py	app.py is executed.	Check if the line model = joblib.load("prod_random_forest.pkl") in app.py executes without raising an exception.	Flask app starts, confirming error-free loading of the trained model file.	App starts, serves index.html without errors loading prod_random_forest.pkl	Pass

WB_02	Input DataFrame Creation in app.py	User submits the form on index.html via a POST request to "/".	In app.py's POST route, verify the pandas DataFrame created from form inputs has column names matching the model's expected feature names.	A pandas DataFrame is created with columns matching the model's input features, ensuring correct data types (numerical as numeric, categorical as object).	Flask POST route's DataFrame has 'longitude', 'latitude', 'ocean_proximity' with correct data types from form inputs.	Pass
WB_03	Prediction Functionality in app.py	A valid DataFrame data has been created in the POST route of app.py.	Run prediction = model.predict(data) to confirm the model's predict method processes the DataFrame and returns a numerical prediction.	The predict method is successfully called without errors, and a numerical prediction for the house price is generated [Successful prediction generation].	When a valid form is submitted, the prediction variable in the Flask POST route holds a numerical value representing the predicted house price.	Pass
WB_04	Preprocessing Pipeline in train_model.py	train_model.py is executed.	Check ColumnTransformer for numerical (SimpleImputer, StandardScaler), categorical (SimpleImputer, OneHotEncoder), and Similarity4Cluster (geospatial) steps.	ColumnTransformer correctly defines pipelines for numerical, categorical, and geospatial features.	train_model.py's ColumnTransformer includes num_pipeline (SimpleImputer, StandardScaler), cat_pipeline (SimpleImputer, OneHotEncoder for 'ocean_proximity'), and Similarity4Cluster for ['latitude', 'longitude'].	Pass

WB_05	Similarity4Cluster Transformer Logic	train_model.py is executed.	Instantiate Similarity4Cluster (n_clusters=10, gamma=0.1, random_state=42), fit and transform latitude/longitude data, verifying output as similarity score array for 10 cluster centers.	Similarity4Cluster fits KMeans, transforms data into similarity scores, outputting an array (samples × clusters).	Similarity4Cluster transforms sample data into a NumPy array (samples, 10) with coordinate similarity scores.	Pass
-------	--------------------------------------	-----------------------------	---	---	---	------

Table 6.5.2 [Whitebox TestCases & Results]

CHAPTER 7

Conclusion and Future work

7.1 Conclusion

This work developed a machine learning-based house price prediction model through a comprehensive process. It began with analyzing a housing dataset, identifying missing values, skewed features, and outliers. Stratified sampling and income categorization ensured representative data. Correlation analysis revealed median income as a strong predictor of house value, with weaker or negative relationships for other features. Data preparation involved handling missing values, scaling, and transforming skewed features using custom transformers integrated into scikit-learn pipelines. Outlier management and feature generation (e.g., ratios, logs, clustering) enhanced data quality. Various regression models were evaluated, with RandomForestRegressor selected for its high R^2 and low RMSE, validated through cross-validation. Feature importance analysis provided insights, and the trained model was saved for deployment via a user-friendly web interface.

7.2 Future work

Future work includes model-based imputation, advanced feature engineering, deep learning models, improved outlier detection, modeling market segments, robust web app enhancements, and real-time model monitoring with retraining.

References

- Akshay, S. (2024). Real Estate Price Prediction using ANN. Indian Scientific Journal Of Research In Engineering And Management,.
- Denny, J. C. (2024). Application of Feature Engineering Techniques and Machine Learning Algorithms for Property Price Prediction.
- Fatbardha, M. (2024). House Price Prediction Using Machine Learning and Artificial Intelligence. Design of Single Chip Microcomputer Control System for Stepping Motor.
- Gangadharayya, V. A. (2024). House Price Prediction Using Machine Learning. Indian Scientific Journal Of Research In Engineering And Management.
- Sahil, P. -. (2024). House Price Prediction. International Journal For Multidisciplinary Research.
- Sookhyun, K. S.-y. (2024). The Effect of Rapid Integrated Planning Improvement Project on Real Estate Price: A Case Study of Cheongpa District 2 in Yongsan-gu, Seoul.
- Zirui, H. (2024). Research on Housing Price Prediction Based on Machine Learning. Highlights in Science Engineering and Technology.